



2. Python: Formatting

"You don't just write code. You also decorate it"

Previous Section:

1. Python: Basic Syntaxes

Writing Python code is not only about making programs work. It is also about making the output clean, readable, and pleasant to look at. Imagine printing hundreds of values onto the screen without any formatting at all—everything would look messy and difficult to understand.

Formatting allows us to organize information properly. We can align text, insert variables into sentences, create tables, and control how values appear on the screen. A well-formatted output makes programs feel more professional and easier to read.

Python provides several ways to format strings and outputs. Some methods are older and simpler, while others are more modern and flexible. In this section, we will explore the most common formatting techniques in Python, starting from the basics and gradually moving toward the modern `f-string` approach.

1. Common Format Techniques

The simplest form of formatting in Python is separating values with commas inside the `print()` function.

```
name = "Alice"
age = 21

print("My name is", name, "and I am", age, "years old.")
```

My name is Alice and I am 21 years old.

This method is easy to use because Python automatically inserts spaces between values. It is simple and beginner-friendly, although it may become inconvenient when dealing with more complex formatting.

Another traditional method uses the `%` operator, inspired by formatting styles from the C programming language.

```
print("My name is %s and I am %d years old!" % ('Michael',
21))
print("This is %s and she is learning %s" % ('Alice', 'Python'))
```

My name is Michael and I am 21 years old!

This is Alice and she is learning Python

Here:

- `%s` represents a string
- `%d` represents an integer

This approach was extremely popular in older Python code and is still found in many projects today.

You can also combine strings using the `+` operator.

```
name = "Johnny"
print("Hello " + name)
```

Hello Johnny

This method works, but it can become difficult to read when many variables are involved. You also need to manually convert non-string values into strings using `str()`.

```
age = 20
print("I am " + str(age) + " years old.")
```

I am 20 years old.

Because of these limitations, Python later introduced more flexible formatting systems.

2. The `format()` Method

The `format()` method provides a cleaner and more organized way to insert values into strings. Instead of manually combining strings together, we place `{}` placeholders inside the text and let Python replace them with values.

```
name = "Maria"
age, major = 23, 'IT'

print("Hello! My name is {} and my age is {} years".format(
    name, age))
print("And my major is {}".format(major))
```

Hello! My name is Maria and my age is 23 years
And my major is IT

This approach is more readable and easier to manage, especially when many variables are involved.

One powerful feature of `format()` is alignment formatting. This is extremely useful when displaying tables or structured outputs.

```
name = ["A", "B", "C", "D", "E", "F"]
age = [21, 19, 19, 18, 19, 20]

for i in range(len(name)):
    print('{} {}'.format(name[i], age[i]))
```

A 21
B 19
C 19
D 18
E 19
F 20

The output above works, but the alignment may look inconsistent. To improve readability, we can define spacing widths.

```
name = ["A", "B", "C", "D", "E", "F"]
age = [21, 19, 19, 18, 19, 20]

for i in range(len(name)):
    print('{:<10} {:>15}'.format(name[i], age[i]))
```

A 21
B 19
C 19
D 18
E 19
F 20

Here:

- `{:>n}` → right-aligns text in a field of width `n`
- `{:<n}` → left-aligns text in a field of width `n`
- `{:^n}` → centers text in a field of width `n`

This makes the output cleaner and much easier to read when displaying multiple rows of information.

3. The `f-string`

The `f-string` (formatted string literal) is considered the modern and most convenient formatting method in Python.

To create one, simply place the letter `f` before a string. Variables and expressions can then be inserted directly using `{}`.

```
name = 'Johnny'

print(f'Hello {name}.')
print(f'The value is {10 + 2}.')
```

Hello Johnny.
The value is 12.

Unlike older methods, `f-strings` are shorter, cleaner, and easier to read. You do not need `%`, `.format()`, or long string concatenations.

```
name = "Maria"
age, major = 23, 'IT'
height, weight = 178, 67

print(f"Hello, I am {name}, {age} years old, and my major is {major}")
print(f"My height and weight is ({height}, {weight})")
```

Hello, I am Maria, 23 years old, and my major is IT
My height and weight is (178, 67)

One major advantage of `f-strings` is that they can directly evaluate expressions inside the braces.

```
x = 10
y = 5

print(f"The sum of x and y is {x + y}")
```

The sum of x and y is 15

But `f-strings` become even more powerful when combined with format specifiers. These allow us to control how numbers appear in the output.

Some common formatting specifiers include:

- `.f` → Shows a floating-point number with decimal places
- `d` → Decimal (integer) format specifier
- `.e` → Displays the number in exponential (scientific) form
- `.0%` → Multiplies the number by 100 and displays it as a percentage

For example:

```
num = 123.456789
# .2f means round the value to 2 decimal places
print(f'{num:.2f}')
```

123.46

```
num = 1000000.00
print(f'{num:,.2f}')
```

1,000,000.00

```
discount = 0.5
print(f'{discount:.0%}')
```

50%

```
num = 123456789
print(f'{num:,d}')
```

123,456,789

```
num = 12345.6789
print(f'{num:.2e}')
```

1.23e+04

These formatting techniques are especially useful when displaying prices, percentages, scientific values, reports, or large datasets.

Because of their simplicity and readability, `f-strings` are now the preferred formatting style in modern Python programming.

So if you are wondering which formatting method you should use most of the time → the answer is usually `f-strings`. They reduce clutter, improve readability, and make your code feel much more natural to write.

4. Escape Characters

Sometimes, formatting text is not just about inserting variables into strings. We also need precise control over how text appears on the screen. This is where escape characters become extremely useful.

Escape characters are special character combinations that begin with a backslash `\`. They allow us to create line breaks, tab spaces, overwrite text, and even remove characters directly inside a string.

One of the most common escape characters is `\n`, which creates a new line.

```
print("This is: \nPython Programming")
```

This is:
Python Programming

This moves `"Python Programming"` onto the next line.

You can also use `\n` to create spacing between outputs.

```
print("This is Python Programming")  
print("\nBy Alice Brants")
```

This is Python Programming

By Alice Brants

Another useful escape character is `\t`, which inserts a tab space.

```
print("This is:\tPython Programming")
print("\tThis is: Python Programming")
print("This\tis:\tPython\nProgramming")
```

This is: Python Programming
 This is: Python Programming
 This is: Python
 Programming

You can even combine multiple tab spaces and line breaks together.

```
print("This is:\t\tPython Programming")
print("This is:\n\nPython Programming")
print("This is:" + '\t' * 5 + "Python Programming")
```

This is: Python Programming
 This is:

 Python Programming
 This is: Python Programming



Another way you can create spaces between output is `print()`. But this is definitely one of those "quick and dirty" habits that can clutter up your code fast.

The `\r` escape character behaves differently. It moves the cursor back to the beginning of the line, causing the following text to overwrite the earlier part.

```
print("This is:Python\rProgramming")
print("This is: \rPython Programming")
```

Programming
 Python Programming


```
print("This is not Python\r")
print("This is \n not\rPython")
print("This is not\r\tPython")
```

This is not Python
This is
Python
 Python

Another interesting escape character is `\b`, which removes the character immediately before it.

```
print("This is:Python\bProgramming")
print("This is: \tPython\bProgramming")
```

This is:PythoProgramming
This is: PythoProgramming

More examples:

```
print("This is not Python\b")
print("This\bis \n not\rPython")
print("This is not\r\tPython\b")
```

This is not Pytho
Thiis
Python
 Pytho

Escape characters may seem small, but they are extremely useful when formatting outputs, designing console interfaces, or improving readability inside terminal applications.

5. Comments

A comment is a programmer-readable explanation written directly inside the source code. Comments are ignored by Python during execution, meaning they exist purely for humans to read and understand the code better.

Comments are important because they explain what a section of code does, making programs easier to maintain and easier for others to understand.

In Python, a single-line comment uses the `#` symbol.

```
# This program displays Welcome to Python
print("Welcome to Python")
print("Python is fun")
```

Welcome to Python
Python is fun

For larger explanations, Python programmers sometimes use triple quotation marks `''' '''` to create multi-line comments or documentation blocks.

```
''' This program displays Welcome to Python and
Python is fun
'''

print("Welcome to Python")
print("Python is fun")
```

Welcome to Python
Python is fun

Comments are technically optional, but in real-world programming they are highly recommended. A well-commented program is easier to debug, easier to improve, and much easier for other programmers to follow.

Think of comments as notes left behind by the programmer. Without them, understanding large programs can become extremely difficult.

Summary

This section introduced some of the most important formatting techniques in Python.

We started with simple formatting methods such as commas, the `%` operator, and string concatenation using `+`. These techniques are useful for basic

outputs, but they can become difficult to manage in larger programs.

Next, we explored the `format()` method, which provides cleaner and more flexible formatting. It allows us to insert values into placeholders and even align text into structured tables.

After that, we moved into modern Python formatting with `f-strings`. These are currently the preferred formatting style because they are shorter, more readable, and extremely powerful. We also learned how format specifiers can control decimal places, percentages, commas, and scientific notation.

Then, we explored escape characters such as `\n`, `\t`, `\r`, and `\b`, which help control the appearance of text more precisely inside the console.

Finally, we discussed comments, one of the most important habits in programming. Comments allow programmers to explain their code, making programs easier to understand, maintain, and improve over time.

Formatting may seem like a small topic at first, but it plays a major role in making programs readable, organized, and professional.

References

<https://www.geeksforgeeks.org/python/python-string-format-method/>

<https://www.geeksforgeeks.org/python/python-escape-characters/>

<https://www.geeksforgeeks.org/python/python-comments/>

Next Section:

 [3. Python: Data Types](#)